Chương 3 (Phần 1): Phát hiện lỗ hổng phần mềm – Phân tích tĩnh

3.1 Kiểm chứng & thẩm định – 3.2 Soundness/Completeness – 3.3 Static/Dynamic/Debug – 3.4 SAT/SMT

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nhắc lại Chương 1, 2

- Chương 1: khái niệm bảo mật, Kripke structure, testing/verification cơ bản
- Chương 2: áp dụng vào C – quản lý bộ nhớ, alignment, công cụ static analysis thực tế (cppcheck, clang-tidy)
- Chương 3 sẽ đi sâu vào **nền tảng lý thuyết** của phân tích tĩnh: soundness/completeness, SAT/SMT, và Bounded Model Checking – công nghệ đứng sau các công cụ đã dùng ở Chương 2

Nội dung Phần 1

1. 3.1 Giới thiệu về kiểm chứng và thẩm định phần mềm
2. 3.2 Hiểu về tính chính xác và đầy đủ (Soundness & Completeness)
3. 3.3 Sự khác biệt giữa phân tích tĩnh, mô phỏng kiểm thử và gỡ lỗi
4. 3.4 SAT và SMT Solving

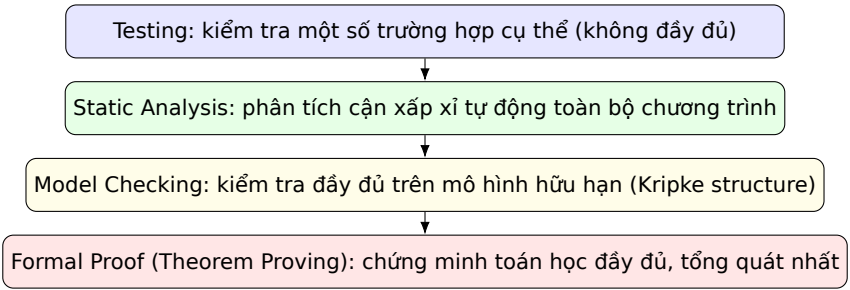
Verification và Validation – Nhắc lại và mở rộng

Định nghĩa

Verification: “Are we building the product right?” – sản phẩm có đúng theo đặc tả không.
Validation: “Are we building the right product?” – sản phẩm có đáp ứng đúng nhu cầu thực tế không.

Chương 3 tập trung vào **Verification** – cụ thể là các kỹ thuật *tự động, hình thức* để chứng minh chương trình thỏa mãn đặc tả.

Phổ các kỹ thuật kiểm chứng



Càng đi xuống, độ tin cậy của kết quả càng cao nhưng chi phí tính toán/công sức càng lớn.

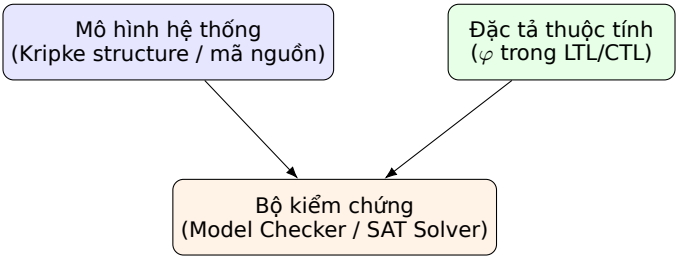
Vì sao cần kiểm chứng tự động thay vì chỉ testing?

- Đã học ở Chương 1: testing không thể chứng minh *không có lỗi* – chỉ chứng minh sự tồn tại của lỗi qua các trường hợp cụ thể
- Với phần mềm an toàn cao (hàng không, y tế, tài chính), cần **bằng chứng hình thức** rằng một số thuộc tính quan trọng luôn đúng
- Kiểm chứng tự động (automated verification) cung cấp mức đảm bảo cao hơn testing, với chi phí thấp hơn formal proof thủ công hoàn toàn

Property Specification – Đặc tả thuộc tính cần kiểm chứng

- Trước khi kiểm chứng, cần **đặc tả rõ ràng** thuộc tính cần chứng minh, thường dưới dạng logic hình thức
- Ví dụ (nhắc lại LTL/CTL từ Chương 1): $\mathbf{AG}\,\neg\text{crash}$ (không bao giờ crash), $\mathbf{AG}(\text{malloc_called} \rightarrow \mathbf{AF}\,\text{free_called})$ (mọi malloc cuối cùng đều có free tương ứng)
- Chất lượng của kết quả kiểm chứng phụ thuộc trực tiếp vào chất lượng đặc tả – “garbage in, garbage out”

Ba thành phần của bài toán kiểm chứng



Đầu ra: **”Đúng”** (kèm chứng minh) hoặc **”Sai”** (kèm counterexample cụ thể) – đã giới thiệu khái niệm này ở Chương 1.

Verification vs. Testing: Bảng so sánh

	Testing	**Formal Verification**
Phạm vi bao phủ	Một số input cụ thể	Toàn bộ không gian trạng thái (trong giới hạn mô hình)
Kết quả khi đúng	”Chưa tìm thấy lỗi”	”Đã chứng minh không có lỗi”
Chi phí	Tương đối thấp	Cao hơn, đặc biệt với hệ thống lớn

Ứng dụng thực tế của Formal Verification

- **Vi xử lý**: Intel dùng model checking để kiểm chứng đơn vị số học dấu phẩy động sau sự cố Pentium FDIV bug (1994)
- **Hàng không**: chuẩn DO-178C yêu cầu bằng chứng kiểm chứng nghiêm ngặt cho phần mềm điều khiển bay
- **Giao thức bảo mật**: công cụ như Tamarin, ProVerif kiểm chứng hình thức giao thức TLS, Signal Protocol
- **Compiler**: CompCert – trình biên dịch C được chứng minh hình thức không có lỗi biên dịch

Bài tập: Xác định loại kiểm chứng phù hợp

Với mỗi tình huống, đề xuất kỹ thuật phù hợp (testing, static analysis, model checking, formal proof):

1. Ứng dụng web thương mại điện tử thông thường, ngân sách hạn chế
2. Firmware điều khiển máy tạo nhịp tim (pacemaker)
3. Giao thức xác thực mới cho hệ thống thanh toán ngân hàng

Đáp án: Xác định loại kiểm chứng phù hợp

1. **Testing** + static analysis cơ bản – đủ với ngân sách hạn chế và rủi ro tương đối thấp
2. **Formal proof / Model checking đầy đủ** – an toàn tính mạng, cần bằng chứng cao nhất có thể
3. **Model checking cho giao thức** (Tamarin/ProVerif) – chuẩn công nghiệp cho việc kiểm chứng giao thức bảo mật

Lịch sử ngắn: Vì sao Formal Verification ra đời

- Những năm 1970-1980: các thảm họa phần mềm (Therac-25 – máy xạ trị gây tử vong do lỗi race condition) thúc đẩy nhu cầu chứng minh hình thức
- Model Checking ra đời (Clarke, Emerson, Sifakis – giải Turing Award 2007) như một phương pháp *tự động hóa* việc kiểm chứng, thay vì chỉ dựa vào chứng minh toán học thủ công
- Ngày nay: model checking và SMT solving là công nghệ nền tảng trong nhiều công cụ công nghiệp (Chương 3 sẽ giới thiệu cách chúng hoạt động)

Case study: Therac-25 (bài học lịch sử)

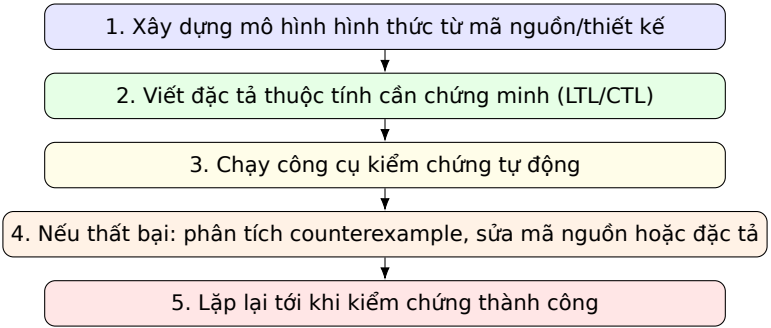
- Máy xạ trị ung thư, phần mềm điều khiển có lỗi *race condition* giữa thao tác nhập liệu nhanh của kỹ thuật viên và cập nhật trạng thái máy
- Lỗi khiến máy đôi khi phát tia xạ liều cực cao, gây tử vong và thương tật nghiêm trọng cho bệnh nhân (1985-1987)
- Đây là một trong những vụ việc thúc đẩy ngành công nghiệp áp dụng kiểm chứng hình thức nghiêm ngặt hơn cho phần mềm an toàn tính mạng (safety-critical)

Property Specification: Phân loại thuộc tính cần kiểm chứng

Nhắc lại và mở rộng từ Chương 1 (Safety/Liveness):

- **Safety**: ”Điều xấu không bao giờ xảy ra” – vd: không bao giờ dùng con trỏ sau khi free
- **Liveness**: ”Điều tốt cuối cùng xảy ra” – vd: mọi tài nguyên cấp phát cuối cùng đều được giải phóng
- **Fairness**: điều kiện phụ trợ đảm bảo mọi tiến trình đều được ”cơ hội” chạy công bằng (quan trọng khi kiểm chứng hệ thống đa luồng)

Quy trình kiểm chứng hình thức trong thực tế



Thách thức thực tế: Xây dựng mô hình từ mã nguồn thực

- Mã nguồn C thực tế có hàng nghìn dòng, nhiều con trỏ, vòng lặp phức tạp – xây dựng Kripke structure thủ công là bất khả thi
- Cần công cụ *tự động trích xuất* mô hình từ mã nguồn (model extraction) – đây chính là vai trò của các công cụ Bounded Model Checking sẽ học ở mục 3.5
- Đánh đổi luôn tồn tại giữa *độ chính xác* của mô hình và *khả năng kiểm chứng được* trong thời gian hợp lý

Bài tập: Viết đặc tả thuộc tính

Với module quản lý phiên đăng nhập đã xây dựng ở Chương 1 (module Python `ch1_models.kripke`), hãy viết một thuộc tính LTL/CTL diễn đạt: ”Không bao giờ có thể chuyển từ trạng thái `locked` sang `authenticated` mà không qua bước xác minh admin (giả sử có trạng thái mới `admin_verify`).”

Đáp án: Viết đặc tả thuộc tính

$$\mathbf{AG}\,(\text{locked} \rightarrow \neg\mathbf{EX}\,\text{authenticated})$$

Diễn giải: ”Tại mọi trạng thái, nếu đang `locked`, thì *không tồn tại* bước chuyển tiếp trực tiếp nào dẫn ngay tới `authenticated`” – bắt buộc phải đi qua trạng thái trung gian (`admin_verify`) trước.

Tổng kết 3.1

- Kiểm chứng (verification) tập trung vào tính đúng đắn theo đặc tả hình thức
- Phổ kỹ thuật: testing $\rightarrow$ static analysis $\rightarrow$ model checking $\rightarrow$ formal proof, độ tin cậy tăng dần
- Bài toán kiểm chứng gồm 3 thành phần: mô hình hệ thống, đặc tả thuộc tính, bộ kiểm chứng tự động

Nhắc lại Soundness và Completeness (từ Chương 1)

Sound (Đúng đắn)	Nếu kỹ thuật báo ”không có lỗi”, thì thực sự không có lỗi – **không bỏ sót**
Complete (Đầy đủ)	Nếu có lỗi thực sự tồn tại, kỹ thuật chắc chắn phát hiện ra – **không báo sai**

Chương 3 sẽ phân tích sâu hơn: *tại sao* không thể có công cụ vừa sound vừa complete cho mọi chương trình, và các đánh đổi thực tế.

Ma trận Sound/Complete: 4 trường hợp

	Complete	**Không Complete**
Sound	Lý tưởng (hiếm, chỉ với bài toán quyết định được, phạm vi hữu hạn)	Bounded Model Checking (không complete ngoài phạm vi bound)
Không Sound	Fuzzing tốt (tìm được nhiều lỗi thật nhưng có thể bỏ sót)	Hầu hết công cụ static analysis thực tế (Chương 2)

Định lý Rice – Vì sao Sound + Complete tuyệt đối là bất khả thi

Định nghĩa

Định lý Rice (1953): Mọi thuộc tính *ngữ nghĩa không tầm thường* (non-trivial semantic property) của chương trình đều **không thể quyết định được** (undecidable) một cách tổng quát.

Ví dụ

”Chương trình này có bao giờ gọi `free(p)` hai lần không?” là một câu hỏi ngữ nghĩa không tầm thường $\Rightarrow$ không tồn tại thuật toán tổng quát trả lời chính xác *Có/Không* cho *mọi* chương trình C.

Hệ quả thực tế của Định lý Rice

- Mọi công cụ phân tích tĩnh thực tế **buộc phải đánh đổi**: hoặc chấp nhận false positive (không complete), hoặc chấp nhận false negative (không sound), hoặc giới hạn phạm vi phân tích (như Bounded Model Checking)
- Đây không phải là hạn chế kỹ thuật tạm thời – đây là **giới hạn toán học nền tảng**, không thể khắc phục bằng công cụ tốt hơn trong tương lai
- Hiểu rõ điều này giúp đặt kỳ vọng đúng đắn khi dùng công cụ phân tích tĩnh (Chương 2) hay model checker (mục 3.5)

Ví dụ: Over-approximation (Xấp xỉ dư – ưu tiên Sound)

Định nghĩa

Over-approximation: công cụ xét *nhiều hơn* tập hành vi thực sự có thể có của chương trình – đảm bảo không bỏ sót lỗi thật (sound), nhưng có thể báo lỗi không thật (false positive).

Ví dụ

Abstract interpretation (đã học ở Chương 2, vd: Astrée) là over-approximation: nếu công cụ nói ”không có lỗi”, chắc chắn đúng; nhưng có thể báo cảnh báo cho trường hợp thực tế không bao giờ xảy ra.

Ví dụ: Under-approximation (Xấp xỉ thiếu – ưu tiên Complete)

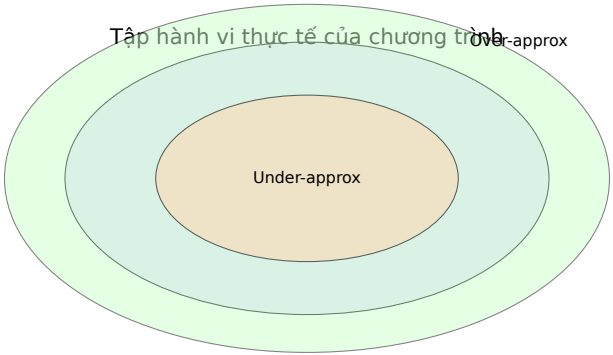
Định nghĩa

Under-approximation: công cụ chỉ xét *một phần* tập hành vi có thể có – mọi lỗi báo ra đều là lỗi thật (complete theo nghĩa không false positive), nhưng có thể bỏ sót lỗi (không sound).

Ví dụ

Testing và Bounded Model Checking (mục 3.5) là under-approximation: nếu tìm thấy lỗi, chắc chắn là lỗi thật; nhưng không tìm thấy lỗi không có nghĩa là không có lỗi (có thể lỗi nằm ngoài phạm vi đã xét).

Sơ đồ trực quan: Over- vs Under-approximation



Over-approximation (vòng ngoài, xanh lá) bao trùm cả tập thực tế; Under-approximation (vòng trong, cam) chỉ nằm gọn bên trong tập thực tế.

Bài tập: Phân loại công cụ theo Sound/Complete

Với mỗi công cụ đã học ở Chương 1, 2, hãy phân loại là over-approximation (ưu tiên sound) hay under-approximation (ưu tiên complete):

1. cppcheck (pattern-based static analysis)
2. Fuzzing (Chương 4)
3. Valgrind (dynamic analysis)

Đáp án: Phân loại công cụ theo Sound/Complete

1. cppcheck: chủ yếu **under-approximation** thực tế (bỏ sót nhiều lỗi phức tạp, dù đôi khi cũng có false positive) – các công cụ thực tế thường không thuần túy một loại
2. Fuzzing: **under-approximation** – lỗi tìm thấy luôn là lỗi thật, nhưng không tìm thấy không đảm bảo an toàn
3. Valgrind: **under-approximation** – chỉ phát hiện lỗi trên đường thực thi *đã chạy qua*, không bao phủ mọi đường có thể

Vì sao Định lý Rice không phải là ”tin xấu” hoàn toàn

- Định lý Rice nói về tính *tổng quát tuyệt đối* – không tồn tại thuật toán đúng cho *mọi* chương trình có thể viết ra
- Nhưng với *một lớp chương trình cụ thể, giới hạn* (vd: không có đệ quy vô hạn, kích thước input giới hạn), bài toán có thể quyết định được
- Đây chính là ý tưởng cốt lõi của **Bounded Model Checking** (mục 3.5): giới hạn phạm vi để đạt được kết quả quyết định được và hữu ích trong thực tế

Halting Problem – Liên hệ với Định lý Rice

Định nghĩa

Bài toán dừng (Halting Problem): không tồn tại thuật toán tổng quát xác định một chương trình bất kỳ có dừng (terminate) hay chạy vô hạn, với input bất kỳ.

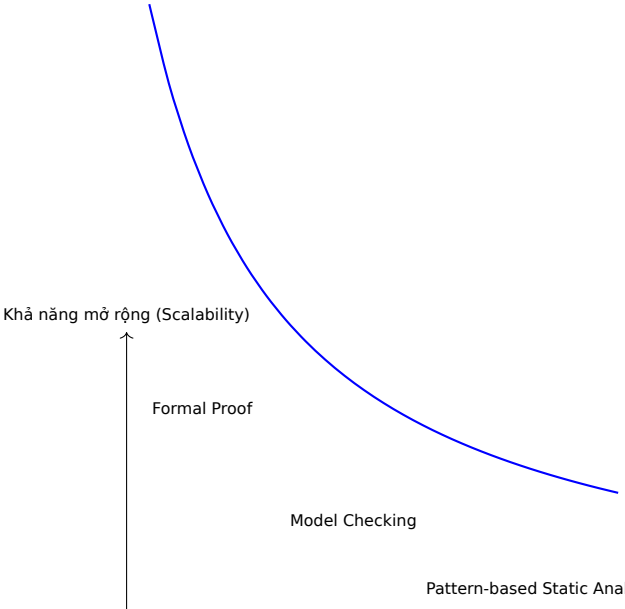
Đây là trường hợp đặc biệt nổi tiếng nhất của Định lý Rice (Turing, 1936) – nền tảng lý thuyết cho toàn bộ giới hạn của kiểm chứng phần mềm tự động.

Ví dụ cụ thể: Vì sao ”không có Use-After-Free” là Undecidable

```
void f(int n) {
    int *p = malloc(sizeof(int));
    if (complex_condition(n)) {
        free(p);
    }
    // p co the da free hay chua, tuy thuoc complex_condition(n)
    *p = 5;  // Co the la use-after-free, tuy n
}
```

Nếu `complex_condition` là một hàm tùy ý phức tạp (có thể chứa cả bài toán dừng bên trong nó), việc xác định *tổng quát* liệu `*p = 5` có phải use-after-free hay không tương đương với giải bài toán dừng.

Trade-off thực tế: Precision vs. Scalability



Bài tập: Phân tích trade-off

Một công ty muốn kiểm chứng *toàn bộ* codebase 5 triệu dòng bằng Formal Proof đầy đủ (theorem proving thủ công). Điều này có khả thi trong thực tế không? Đề xuất chiến lược thay thế.

Đáp án: Phân tích trade-off

- **Không khả thi** – formal proof thủ công cho 5 triệu dòng cần hàng trăm năm-người, không thực tế
- Chiến lược thay thế: áp dụng formal proof/model checking đầy đủ chỉ cho *module lõi quan trọng nhất* (vd: hàm xử lý xác thực, mã hóa), còn lại dùng static analysis tự động + testing + code review (chiến lược phân lớp theo mức độ rủi ro, liên hệ Chương 2 mục 2.1)

Liên hệ Sound/Complete với các công cụ đã học ở Chương 2

Công cụ	**Sound?**	**Complete?**
cppcheck	Không hoàn toàn	Không hoàn toàn
Clang Static Analyzer	Thường hướng sound hơn	Không (có false positive)
Valgrind (dynamic)	Không (chỉ đường đã chạy)	Có (với đường đã chạy)
Astrée (abstract interp.)	Có (cho lớp lỗi đã định nghĩa)	Không (nhiều false positive)

Tổng kết 3.2

- Sound: không bỏ sót lỗi thật; Complete: không báo sai
- Định lý Rice chứng minh không thể có công cụ tổng quát vừa sound vừa complete cho mọi thuộc tính ngữ nghĩa không tầm thường
- Công cụ thực tế phải chọn: over-approximation (sound, có thể false positive) hoặc under-approximation (complete theo nghĩa không false positive, có thể bỏ sót lỗi)

Ba hoạt động dễ nhầm lẫn: Static Analysis, Testing/Simulation, Debugging

- Cả ba đều nhằm mục đích *tìm và hiểu lỗi*, nhưng khác nhau về **thời điểm**, **cách thức**, và **mục tiêu**
- Sinh viên thường nhầm lẫn giữa ”chạy chương trình để debug” và ”kiểm thử” – mục 3.3 làm rõ ranh giới

Static Analysis – Nhắc lại

Định nghĩa

Phân tích mã nguồn *không thực thi* chương trình, thường nhằm phát hiện *lớp lỗi* tiềm ẩn trên toàn bộ (hoặc phần lớn) không gian trạng thái.

Mục tiêu: tìm lỗi **sớm**, **tự động**, bao phủ rộng nhưng có thể kém chính xác (Chương 2).

Định nghĩa

Testing: thực thi chương trình với input cụ thể, so sánh kết quả với oracle mong đợi.
Simulation: chạy một *mô hình* của hệ thống (không phải chính hệ thống thật) để quan sát hành vi, thường dùng khi hệ thống thật khó/tốn kém để thử trực tiếp (vd: mô phỏng mạch điện tử, mô phỏng giao thông).

Mục tiêu: xác nhận hành vi *thực tế* đúng theo mong đợi cho các trường hợp cụ thể đã chọn.

Định nghĩa

Debugging: quá trình *điều tra nguyên nhân gốc rễ* của một lỗi *đã biết* (thường phát hiện qua testing hoặc từ báo cáo người dùng), thường dùng công cụ debugger (gdb, lldb) để theo dõi trạng thái chương trình từng bước.

Khác biệt cốt lõi: debugging xảy ra *sau khi* đã biết có lỗi, mục tiêu là *định vị và hiểu* lỗi, không phải *tìm kiếm* lỗi mới.

Bảng so sánh tổng hợp

	Static Analysis	**Testing/Simulation**	**Debugging**
Thực thi?	Không	Có	Có (từng bước)
Mục tiêu	Tìm lỗi tiềm ẩn	Xác nhận hành vi đúng/sai	Định vị nguyên nhân lỗi đã biết
Thời điểm	Sớm nhất (compile-time)	Sau khi có thể chạy được	Sau khi phát hiện lỗi
Đầu vào	Toàn bộ mã nguồn	Test case cụ thể	Trạng thái lỗi cụ thể

Ví dụ minh họa: Cùng một lỗi, 3 góc nhìn

```
int divide(int a, int b) {
    return a / b;   // Loi tiem an: chia cho 0
}
```

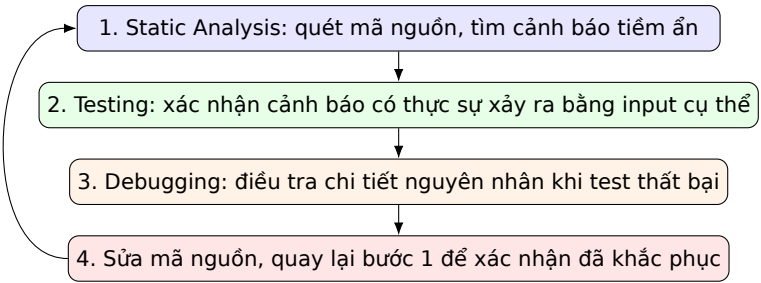
- **Static analysis**: cảnh báo ”có thể chia cho 0” mà *không cần chạy*, dựa trên phân tích mọi giá trị b có thể có
- **Testing**: gọi `divide(10, 0)` và quan sát chương trình crash – xác nhận lỗi *thực sự xảy ra* với input này
- **Debugging**: dùng gdb, đặt breakpoint tại dòng chia, theo dõi giá trị b để hiểu *tại sao* nó bằng 0 trong ngữ cảnh gọi hàm thực tế

Công cụ Debugging cơ bản: GDB

```
gdb ./program
(gdb) break divide
(gdb) run
(gdb) print b
(gdb) backtrace
```

gdb cho phép dừng chương trình tại một điểm cụ thể (breakpoint), xem giá trị biến, và xem chuỗi lời gọi hàm dẫn tới điểm đó (backtrace) – công cụ thiết yếu để *hiểu* lỗi sau khi đã *phát hiện* nó.

Vai trò bổ sung lẫn nhau trong quy trình phát triển



Bài tập: Xác định giai đoạn phù hợp

Với mỗi hoạt động sau, xác định đây là static analysis, testing, hay debugging:

1. Chạy cppcheck trước khi commit mã nguồn
2. Đặt breakpoint tại dòng nghi ngờ và xem giá trị biến từng bước
3. Viết một unit test mới để tái hiện lỗi người dùng báo cáo

Đáp án: Xác định giai đoạn phù hợp

1. **Static Analysis** – không chạy chương trình, quét mã nguồn
2. **Debugging** – điều tra chi tiết nguyên nhân, dùng breakpoint
3. **Testing** – thực thi với input cụ thể để xác nhận hành vi (dù mục đích là tái hiện lỗi, đây vẫn thuộc phạm trù testing)

Trace-based Debugging vs. Interactive Debugging

Interactive	Dùng debugger (gdb) để dừng chương trình và kiểm tra trạng thái trực tiếp, từng bước	Trace-based hữu ích khi lỗi khó tái hiện (heisenbug – lỗi biến mất khi thêm debugger do thay đổi timing), phổ biến trong hệ thống production.
Trace-based	Ghi lại toàn bộ log/trace của chương trình khi chạy, sau đó phân tích lại (không cần dừng chương trình thời gian thực)	

Reverse Debugging (giới thiệu)

- Công cụ debugger hiện đại (rr, gdb với reverse execution) cho phép *chạy lùi* chương trình, xem trạng thái *trước đó* dẫn tới lỗi
- Đặc biệt hữu ích cho lỗi bộ nhớ: thay vì chỉ biết ”con trỏ này bị hỏng ở đây”, có thể lùi lại tìm chính xác *đâu* là nơi ghi đè sai lần đầu tiên
- Kết hợp tốt với AddressSanitizer (Chương 2) – ASan phát hiện lỗi, reverse debugger giúp truy vết nguyên nhân gốc

Logging và Assertion như công cụ Debug thụ động

```c
#ifdef DEBUG
#define LOG(msg, ...) fprintf(stderr, "[DEBUG] " msg "\n", ##__VA_ARGS__)
#else
#define LOG(msg, ...)
#endif

void process(int *data, int len) {
    LOG("processing %d items", len);
    assert(data != NULL);
    // ...
}
```

Logging có kiểm soát (bật/tắt qua `#ifdef`) giúp debug production mà không ảnh hưởng hiệu năng bản release.

Simulation trong bối cảnh hệ thống nhúng/an toàn

- Với hệ thống nhúng (firmware xe hơi, hàng không), việc ”testing” trực tiếp trên phần cứng thật có thể tốn kém hoặc nguy hiểm
- **Hardware-in-the-loop simulation**: chạy phần mềm điều khiển thật, kết nối với mô hình mô phỏng phần cứng (không phải phần cứng thật) để quan sát hành vi an toàn trước khi triển khai thực tế
- Đây là điểm khác biệt quan trọng giữa ”testing” (input rời rạc) và ”simulation” (chạy liên tục theo thời gian mô phỏng)

Bài tập: Phân biệt 3 kỹ thuật qua tình huống

Một kỹ sư phát hiện chương trình thỉnh thoảng bị crash trong môi trường production nhưng không tái hiện được trên máy phát triển. Đề xuất một chiến lược kết hợp cả 3 kỹ thuật (static analysis, testing/simulation, debugging) để xử lý.

Đáp án: Phân biệt 3 kỹ thuật qua tình huống

1. **Static analysis**: quét lại mã nguồn vùng nghi ngờ tìm race condition/lỗi bộ nhớ tiềm ẩn không phụ thuộc input cụ thể
2. **Testing/Simulation**: thử tái hiện bằng cách mô phỏng tải cao (nhiều luồng đồng thời) giống môi trường production hơn máy phát triển
3. **Debugging**: nếu tái hiện được, dùng trace-based debugging hoặc reverse debugging (vì đây có dấu hiệu heisenbug – phụ thuộc timing)

Chi phí và lợi ích: Khi nào ưu tiên kỹ thuật nào

Giai đoạn	Ưu tiên	Lý do
Đang viết code	Static Analysis	Phản hồi tức thì, chi phí thấp nhất
Trước khi release	Testing toàn diện	Xác nhận hành vi thực tế đúng đắn
Sau khi có báo cáo lỗi	Debugging	Cần hiểu nguyên nhân cụ thể đã xảy ra

Tổng kết 3.3

- Static Analysis: không thực thi, tìm lỗi sớm, bao phủ rộng
- Testing/Simulation: thực thi cụ thể, xác nhận hành vi thực tế
- Debugging: điều tra sau khi đã biết có lỗi, dùng công cụ như gdb
- Ba hoạt động bổ sung lẫn nhau trong một quy trình phát triển lặp lại

Bài toán SAT (Boolean Satisfiability)

Định nghĩa

SAT: cho một công thức logic mệnh đề (boolean formula) ϕ, xác định có tồn tại một phép gán giá trị True/False cho các biến sao cho ϕ đúng hay không.

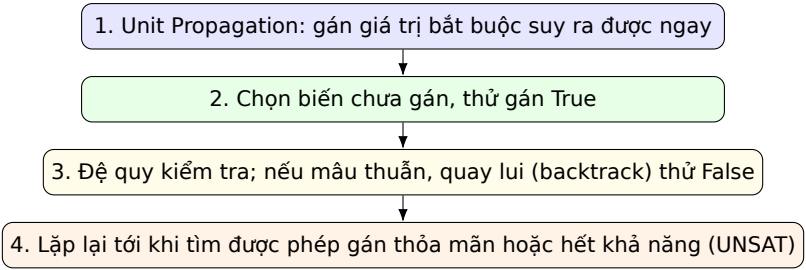
Ví dụ

$\phi = (a \vee b) \wedge (\neg a \vee c) \wedge (\neg b \vee \neg c)$. Đây có SAT không? Thử $a = T, b = F, c = T$: $(T) \wedge (T) \wedge (T) = T$ $\Rightarrow$ SAT (thỏa mãn được).

SAT là bài toán NP-Complete

- SAT là bài toán NP-Complete đầu tiên được chứng minh (Cook-Levin, 1971)
- Về lý thuyết, không có thuật toán nào chạy trong thời gian đa thức cho *mọi* trường hợp (trừ khi P = NP)
- Tuy nhiên, các bộ giải SAT hiện đại (SAT solver) rất hiệu quả *trong thực tế* với hàng triệu biến, nhờ các heuristic thông minh

Thuật toán DPLL – Nền tảng của SAT Solver



DPLL (Davis-Putnam-Logemann-Loveland) là nền tảng cho hầu hết SAT solver hiện đại (MiniSat, Glucose, CaDiCaL...).

Ví dụ chạy DPLL từng bước

$\phi = (a \vee b) \wedge (\neg a \vee c) \wedge (\neg b \vee \neg c)$

- Chọn $a = T$: $(\neg a \vee c)$ yêu cầu $c = T$
- Với $c = T$: $(\neg b \vee \neg c)$ yêu cầu $b = F$
- Kiểm tra $(a \vee b)$: $T \vee F = T \Rightarrow$ tất cả mệnh đề đều đúng
- Kết quả: SAT với $a = T, b = F, c = T$ (khớp với ví dụ đã thử trước)

Định nghĩa

SMT (Satisfiability Modulo Theories): mở rộng SAT để hỗ trợ các **lý thuyết nền** (theories) như số nguyên, mảng, con trỏ, thay vì chỉ biến boolean thuần túy.

Ví dụ

SAT chỉ xử lý được: $a \vee b$. SMT xử lý được: $x + y > 10 \wedge x < 5$ (lý thuyết số nguyên tuyến tính) – gần với điều kiện thực tế trong mã nguồn hơn nhiều.

Các lý thuyết (Theories) phổ biến trong SMT

Lý thuyết	**Ứng dụng**
Linear Arithmetic	Kiểm tra điều kiện số học (vd: `x + y > 10`)
Bit-vectors	Mô phỏng chính xác kiểu số nguyên có kích thước cố định trong C (bao gồm cả tràn số!)
Arrays	Mô hình hóa mảng/con trỏ trong chương trình
Uninterpreted Functions	Trừu tượng hóa hành vi hàm chưa biết chi tiết

SMT Solver phổ biến

- **Z3** (Microsoft Research) – phổ biến nhất, dùng trong nhiều công cụ phân tích mã nguồn công nghiệp
- **CVC5** – mã nguồn mở, hiệu năng cạnh tranh với Z3
- **Yices** – tối ưu cho một số lớp bài toán cụ thể
- Các công cụ trên hỗ trợ chuẩn định dạng chung **SMT-LIB**, giúp chuyển đổi giữa các solver dễ dàng

Ví dụ dùng Z3 (Python API)

```python
from z3 import Int, Solver, sat

x, y = Int('x'), Int('y')
s = Solver()
s.add(x + y > 10)
s.add(x < 5)
if s.check() == sat:
    print(s.model())   # Vi du: x = 4, y = 7
```

Z3 tự động tìm một phép gán x, y thỏa mãn cả hai ràng buộc – đây chính là nền tảng để sinh test case tự động (liên hệ symbolic execution, Chương 1).

Liên hệ SMT với Symbolic Execution (nhắc lại Chương 1)

- Symbolic execution thu thập *path condition* (điều kiện đường đi) dưới dạng công thức logic
- SMT solver kiểm tra path condition đó có *thỏa mãn được* (satisfiable) hay không
- Nếu SAT: solver trả về giá trị cụ thể (model) $\Rightarrow$ chính là **test case tự động** kích hoạt đường thực thi đó
- Nếu UNSAT: đường thực thi đó *không thể xảy ra* với bất kỳ input nào – static analysis có thể loại bỏ false positive dựa trên kết quả này

Ví dụ ứng dụng: Sinh Test Case bằng SMT

```
int classify(int x) {
    if (x > 0 && x < 100) {
        return 1;  // Muc tieu: sinh test case den duoc day
    }
    return 0;
}
```

Path condition cho nhánh `return 1`: $x > 0 \wedge x < 100$. SMT solver giải ra ví dụ $x = 50$ – đây chính là test case tự động sinh ra để đạt **branch coverage** (đã học ở Chương 1) cho nhánh này.

Bài tập: Giải SAT bằng tay

Cho $\phi = (a \vee \neg b) \wedge (b \vee c) \wedge (\neg a \vee \neg c)$. Công thức này có SAT không? Nếu có, tìm một phép gán thỏa mãn.

Đáp án: Giải SAT bằng tay

- Thử $a = F$: mệnh đề 1 $(F \vee \neg b)$ yêu cầu $b = F$
- Với $b = F$: mệnh đề 2 $(F \vee c)$ yêu cầu $c = T$
- Kiểm tra mệnh đề 3 $(\neg a \vee \neg c) = (T \vee F) = T \Rightarrow$ tất cả đúng
- Kết quả: **SAT** với $a = F, b = F, c = T$

Bài tập: Viết ràng buộc SMT

Một hàm C có điều kiện: `if (age >= 18 && age <= 65 && has_license)`. Hãy viết ràng buộc SMT (dùng Python Z3 API) để sinh một bộ giá trị age, has_license thỏa mãn điều kiện này.

Đáp án: Viết ràng buộc SMT

```python
from z3 import Int, Bool, Solver, sat, And

age = Int('age')
has_license = Bool('has_license')
s = Solver()
s.add(And(age >= 18, age <= 65, has_license == True))
if s.check() == sat:
    print(s.model())   # Vi du: age = 18, has_license = True
```

Giới hạn của SAT/SMT Solving

- Dù SAT là NP-Complete, solver hiện đại xử lý tốt hầu hết bài toán *thực tế* (structured, không phải worst-case ngẫu nhiên)
- Vẫn tồn tại bài toán ”khó” khiến solver chạy rất lâu hoặc timeout (đặc biệt với công thức phi tuyến, hoặc quá nhiều biến bit-vector)
- Chất lượng encoding (cách biểu diễn bài toán thành công thức logic) ảnh hưởng lớn tới hiệu năng solver

Conjunctive Normal Form (CNF) – Dạng chuẩn cho SAT

Định nghĩa

CNF: công thức logic dạng *hội của các tuyển* (AND của các OR), vd: $(a \vee b) \wedge (\neg a \vee c)$. Hầu hết SAT solver yêu cầu input ở dạng CNF.

Mọi công thức logic mệnh đề đều có thể chuyển về CNF (có thể tăng kích thước công thức đáng kể) – đây là bước tiền xử lý bắt buộc trước khi đưa vào solver dạng DPLL.

Unit Propagation – Cơ chế tăng tốc quan trọng

Ví dụ

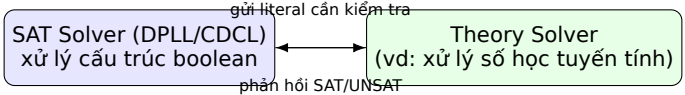
Với mệnh đề $(\neg a \vee c)$, nếu đã biết $a = T$, ta **suy ra ngay** c phải $= T$ (không cần thử cả 2 trường hợp) – đây gọi là *unit propagation*.

Unit propagation giúp SAT solver loại bỏ nhanh chóng nhiều nhánh tìm kiếm không cần thiết, là một trong những lý do chính khiến SAT solver hiện đại hiệu quả hơn nhiều so với brute-force thử mọi tổ hợp.

Conflict-Driven Clause Learning (CDCL) – Cải tiến DPLL hiện đại

- Khi solver gặp mâu thuẫn (conflict), thay vì chỉ quay lui đơn giản, CDCL **phân tích nguyên nhân** mâu thuẫn và học ra một *mệnh đề mới* (learned clause) để tránh lặp lại sai lầm tương tự
- Đây là cải tiến quan trọng nhất giúp SAT solver hiện đại (MiniSat, Glucose trở đi) xử lý được bài toán hàng triệu biến trong thực tế
- CDCL cũng là nền tảng của hầu hết SMT solver hiện đại (kết hợp DPLL/CDCL cho phần boolean + bộ giải chuyên biệt cho từng lý thuyết)

Kiến trúc SMT Solver: DPLL(T)



Kiến trúc **DPLL(T)**: SAT solver xử lý phần cấu trúc logic boolean, ”ủy quyền” các ràng buộc thuộc lý thuyết cụ thể (số học, mảng...) cho theory solver chuyên biệt – kết hợp hiệu quả của cả hai.

Bit-vector Theory – Mô hình hóa chính xác kiểu C

```python
from z3 import BitVec, Solver, sat

x = BitVec('x', 8)   # unsigned char, 8-bit
s = Solver()
s.add(x + 10 == 5)   # Tran so: x = 251 (251 + 10 = 261 mod 256 = 5)
if s.check() == sat:
    print(s.model())
```

Bit-vector theory mô hình hóa *chính xác* hành vi tràn số (wrap-around) của kiểu số nguyên C có kích thước cố định – quan trọng để phát hiện lỗi integer overflow đã học ở Chương 2 một cách hình thức.

Array Theory – Mô hình hóa Mảng/Bộ nhớ

```python
from z3 import Array, IntSort, Solver, Store, Select

mem = Array('mem', IntSort(), IntSort())
mem2 = Store(mem, 0, 42)     # mem2 = mem voi mem[0] = 42
s = Solver()
s.add(Select(mem2, 0) == 42)  # Doc lai gia tri vua ghi
print(s.check())              # sat
```

`Store`/`Select` mô hình hóa chính xác thao tác ghi/đọc mảng – nền tảng để mô hình hóa **Memory Model** sẽ học ở mục 3.6.

Quantifiers trong SMT (giới thiệu nâng cao)

- SMT cơ bản chỉ xử lý công thức *không có lượng từ* (quantifier-free) – hiệu quả và luôn quyết định được (decidable) cho nhiều lý thuyết
- Thêm lượng từ $\forall, \exists$ (vd: "$\forall i, 0 \leq i < n \rightarrow arr[i] \geq 0$") làm bài toán trở nên khó hơn nhiều, có thể *không quyết định được* với một số lý thuyết
- Công cụ thực tế thường dùng heuristic (e-matching, triggers) để xử lý lượng từ hiệu quả trong đa số trường hợp thực tế, dù không đảm bảo tổng quát

Ứng dụng SMT ngoài phân tích mã nguồn

- **Lập lịch (Scheduling)**: tìm phân công tài nguyên thỏa mãn ràng buộc phức tạp
- **Xác minh phần cứng (Hardware Verification)**: kiểm chứng mạch số không có lỗi logic
- **Giải bài toán tối ưu (kết hợp với hàm mục tiêu)**: SMT mở rộng thành **OMT** (Optimization Modulo Theories)
- Trong phạm vi học phần, ta tập trung vào ứng dụng cho **kiểm chứng phần mềm**

Bài tập: Phân tích công thức CNF

Chuyển công thức sau về CNF: $(a \wedge b) \vee c$

Đáp án: Phân tích công thức CNF

Áp dụng luật phân phối: $(a \wedge b) \vee c \equiv (a \vee c) \wedge (b \vee c)$
Đây đã là dạng CNF (hội của 2 mệnh đề tuyển). Với công thức phức tạp hơn, việc chuyển CNF có thể làm tăng kích thước theo cấp số mũ – đây là lý do các công cụ thực tế dùng kỹ thuật *Tseitin transformation* để chuyển đổi hiệu quả hơn (thêm biến phụ thay vì mở rộng trực tiếp).

Bài tập: Mô hình hóa bằng Bit-vector

Một biến `unsigned char count` trong C bắt đầu từ 250 và được cộng thêm 10. Hãy viết ràng buộc Z3 (bit-vector 8-bit) để tìm giá trị kết quả sau phép cộng, xác nhận hiện tượng tràn số đã học ở Chương 2.

Đáp án: Mô hình hóa bằng Bit-vector

```python
from z3 import BitVecVal, simplify

count = BitVecVal(250, 8)
result = simplify(count + 10)
print(result)   # Ket qua: 4 (250 + 10 = 260 mod 256 = 4)
```

Kết quả khớp chính xác với ví dụ tràn số đã học ở Chương 2 (mục 2.1) – xác nhận bit-vector theory mô hình hóa đúng ngữ nghĩa số nguyên có kích thước cố định của C.

Chuẩn bị dự án Python: module ch3_staticsat

Module `ch3_staticsat` (thực hành buổi sau) sẽ:

- Cài đặt một **SAT solver mini** bằng thuật toán DPLL cơ bản (không cần CDCL đầy đủ) để hiểu rõ cơ chế
- Dùng thư viện `z3-solver` (Python binding chính thức của Z3) để giải các ràng buộc SMT thực tế
- Xây dựng nền tảng cho **Bounded Model Checker mini** ở phần thực hành tiếp theo (mục 3.5)

Tổng kết 3.4

- SAT: bài toán quyết định thỏa mãn công thức mệnh đề, NP-Complete, giải bằng DPLL
- SMT: mở rộng SAT với các lý thuyết (số học, mảng, bit-vector) – gần với chương trình thực tế hơn
- Z3, CVC5 là các SMT solver phổ biến, có Python API dễ sử dụng
- SMT là nền tảng của symbolic execution và sẽ là nền tảng của Bounded Model Checking ở Phần 2

So sánh SAT Solver và SMT Solver

	SAT Solver	**SMT Solver**
Biến	Chỉ boolean	Số nguyên, mảng, bit-vector...
Độ biểu đạt	Thấp hơn	Cao hơn, gần với mã nguồn thực tế
Ứng dụng chính	Bài toán tổ hợp thuần túy	Kiểm chứng phần mềm, symbolic execution

Timeline lịch sử SAT/SMT (tóm tắt)

- 1971: Cook-Levin chứng minh SAT là NP-Complete
- 1962: Thuật toán DPLL ra đời (trước cả khi NP-Complete được định nghĩa!)
- Thập niên 2000: CDCL cách mạng hóa hiệu năng SAT solver thực tế
- 2007: Z3 ra mắt (Microsoft Research), trở thành SMT solver phổ biến nhất hiện nay

Bài tập tổng hợp: Từ mã nguồn tới SMT

```
int check(int a, int b, int c) {
    if (a + b == c && a > 0 && b > 0) {
        return 1;  // Nhanh muc tieu
    }
    return 0;
}
```

Hãy viết ràng buộc Z3 (Python) để sinh tự động một bộ giá trị `a, b, c` đạt tới nhánh `return 1`.

Đáp án: Từ mã nguồn tới SMT

```python
from z3 import Int, Solver, And, sat

a, b, c = Int('a'), Int('b'), Int('c')
s = Solver()
s.add(And(a + b == c, a > 0, b > 0))
if s.check() == sat:
    print(s.model())  # Vi du: a=1, b=1, c=2
```

Đây chính là quy trình cốt lõi của **symbolic execution có hỗ trợ SMT** – tự động sinh test case đạt được độ bao phủ nhánh mong muốn mà không cần thử ngẫu nhiên.

Câu hỏi ôn tập tổng hợp Phần 1 (1/2)

1. Phân biệt Verification và Validation, cho ví dụ
2. Vì sao Định lý Rice khiến ”công cụ hoàn hảo” là bất khả thi?
3. SAT khác SMT ở điểm cốt lõi nào?

Câu hỏi ôn tập tổng hợp Phần 1 (2/2)

4. Unit propagation trong DPLL hoạt động như thế nào?
5. Debugging khác Testing ở điểm nào về mục tiêu và thời điểm?
6. Bit-vector theory trong SMT giải quyết vấn đề gì mà số nguyên toán học thông thường không giải quyết được?

Đáp án gợi ý

1. Verification: đúng theo đặc tả; Validation: đúng theo nhu cầu người dùng. Vd: hệ thống tính thuế đúng công thức (verification) nhưng công thức đó không phải điều khách hàng cần (thiếu validation)
2. Vì không tồn tại thuật toán tổng quát quyết định được mọi thuộc tính ngữ nghĩa không tầm thường cho mọi chương trình
3. SAT chỉ xử lý biến boolean thuần túy; SMT mở rộng với các lý thuyết như số học, mảng, bit-vector – biểu đạt gần với chương trình thực tế hơn

Tài liệu tham khảo thêm cho Phần 1

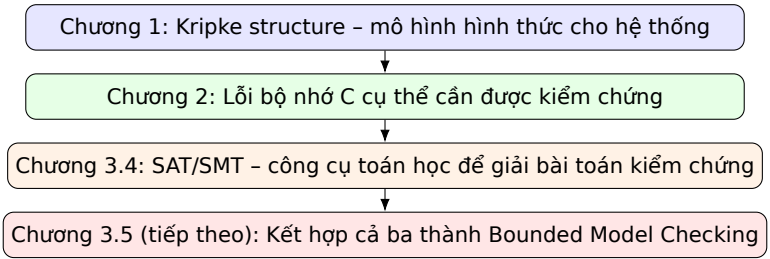
- Clarke, Grumberg, Peled, *Model Checking* – chương về SAT/SMT nền tảng
- Z3 Tutorial chính thức (Microsoft Research)
- Biere et al., *Handbook of Satisfiability* – tài liệu toàn diện về SAT solving

Danh sách kỹ năng thực hành đã đạt được (Phần 1)

Sau Phần 1 của Chương 3, sinh viên đã có thể:

- Phân biệt và giải thích rõ soundness/completeness cho một công cụ bất kỳ
- Giải một bài toán SAT nhỏ bằng tay theo thuật toán DPLL
- Viết ràng buộc SMT cơ bản bằng Python Z3 API và diễn giải kết quả
- Phân biệt rõ static analysis, testing/simulation, và debugging trong tình huống thực tế

Kết nối tổng thể: Từ Chương 1 đến Chương 3



Câu hỏi thảo luận mở

Nếu một SMT solver báo UNSAT cho path condition của một nhánh mã nguồn, điều đó có ý nghĩa gì đối với việc một static analyzer từng cảnh báo ”có thể có lỗi” tại nhánh đó?

Gợi ý trả lời

- UNSAT nghĩa là **không tồn tại** bất kỳ giá trị input nào khiến nhánh đó được thực thi
- Do đó, cảnh báo của static analyzer tại nhánh này chắc chắn là **false positive** – nhánh đó *không bao giờ* thực sự chạy được
- Đây chính là cách các công cụ hiện đại dùng SMT solver để *tự động loại bỏ* một phần đáng kể false positive sinh ra từ static analysis (over-approximation), kết hợp ưu điểm của cả hai kỹ thuật

Vai trò Phần 1 trong bài tập lớn

Nếu bài tập lớn liên quan tới kiểm chứng hình thức, sinh viên nên thử dùng Z3 Python API để tự viết một vài ràng buộc kiểm tra thuộc tính đơn giản của chương trình mình chọn – đây là bước khởi đầu tốt trước khi tiếp cận Bounded Model Checking đầy đủ ở Phần 2.

Điểm cần lưu ý khi ôn tập Phần 1

- Nắm chắc định nghĩa và ví dụ minh họa của soundness/completeness – đây là khái niệm xuyên suốt cả chương
- Luyện tập giải SAT bằng tay ít nhất 2-3 bài để hiểu cơ chế DPLL trước khi dùng solver tự động
- Phân biệt rõ ba hoạt động ở mục 3.3 qua các tình huống cụ thể, không chỉ định nghĩa suông

Hỏi đáp

Câu hỏi?

Chuyển tiếp sang Phần 2

Phần 2 của Chương 3 sẽ trình bày:

- 3.5 Kiểm chứng mô hình phần mềm (Bounded Model Checking) – kết hợp Kripke structure (Chương 1) và SMT (mục 3.4)
- 3.6 Mô hình bộ nhớ (Memory Model)
- 3.7 Sắp xếp mô hình bộ nhớ (Aligned Memory Model) – hình thức hóa alignment đã học ở Chương 2